Handbook

Build guide for an HRMS: a Headcount-style HR management system

Django + Django REST Framework, React (Vite, TypeScript), PostgreSQL
Built with the DeepSeek harness: one phase per session, one prompt per phase

Phase	What you get	Rough time with an agent
0. Setup	Django + React + Postgres running together	Half a day
1. Auth and roles	Login and four roles	1-2 days
2. Teams and employees	People records, invites, audit log	2-3 days
3. Time off	Leave request, approval, balances, calendar	3-5 days
4. Polish and demo	Clean UI, demo data, README	2 days

Times are rough and assume you review what the agent writes.

1. What we are building

A small HR system for one company. HR manages employees and teams, employees see their own profile and request time off, managers approve leave for their team. Everything is stored in PostgreSQL and every important change is logged.

In scope

- Login and four roles: admin, hr, manager, employee.
- Employees: create, edit, deactivate, search, filter. Teams with a lead.
- Time off: leave types, yearly balances, request, approve, reject, cancel, team calendar.
- Audit log of who changed what.
- A good-looking dashboard UI.

Out of scope (for now)

- Payroll, recruiting, attendance tracking, performance reviews, mobile app.
- Multi-company support and SSO.

2. Tech stack

Part	Choice	Why
Backend	Django 5 + Django REST Framework	Auth, ORM, migrations and admin come built in
Frontend	React 18 + Vite + TypeScript	Fast to build; no second server needed
UI	Tailwind + shadcn/ui	Clean dashboard look with little custom CSS
Data fetching	TanStack Query + React Router	Standard, agents write it well
Forms	react-hook-form + zod	Validation with inline errors
Database	PostgreSQL 16	Constraints and transactions for leave logic
Auth	Django sessions + CSRF	Simplest and safe when React is same-origin
Dev setup	Docker for Postgres only	Run Django and Vite directly; fewer moving parts
Tests	pytest-django	Only for permissions and leave logic

Not used: FastAPI, Next.js, any other UI kit.

3. Roles

Role	Can do
admin	Everything, including roles and settings
hr	Manage all employees and teams, leave types, see all leave requests
manager	See own team, approve or reject team leave, view team calendar
employee	View own profile, edit phone, request leave, see own balance

4. Data model

Table	Key fields
User	email (unique), password, is_active, role
Team	name (unique), lead (nullable link to Employee)
Employee	user (1-to-1), first_name, last_name, phone, job_title, team, manager (self link), start_date, employment_type, status
LeaveType	name, annual_days, is_paid, requires_approval, carry_over_max
LeaveBalance	employee, leave_type, year, allocated, used (unique per employee + type + year)
LeaveRequest	employee, leave_type, start_date, end_date, half_day, days, status (pending / approved / rejected / cancelled), reviewer, review_comment
Holiday	date, name
AuditLog	actor, action, model, object_id, changes (JSON), created_at

Database rules: end_date must not be before start_date, and one employee cannot have overlapping non-cancelled leave (a Postgres exclusion constraint).

5. Rules that must never break

- Permissions are enforced in the API. Every endpoint has a permission class and a queryset scoped by role: employee sees own records, manager sees own team, hr and admin see all. Hiding a button in React is never enough.
- Leave logic lives in one file, timeoff/services.py, not in views or the frontend.
- Changing a balance or a request status happens inside one database transaction with the balance row locked.
- Working days count Monday to Friday minus holidays; a half day counts as 0.5.
- Approve deducts the balance. Reject or cancel of an approved request gives it back.
- Employees are deactivated, never deleted. Inactive users cannot log in.
- Approver is the employee's manager, or HR if there is none.

6. Pages and API

Page	Who sees it	API it uses
Login	Everyone	POST /api/v1/auth/login/
Employees list and detail	hr, admin (managers: own team)	/api/v1/employees/
Teams	hr, admin	/api/v1/teams/
My time off	Everyone	/leave-balances/, /leave-requests/
Approvals	manager, hr, admin	/leave-requests/{id}/approve|reject/
Team calendar	Everyone (scoped)	/api/v1/leave-calendar/
Leave types and holidays	hr, admin	/leave-types/ (or Django admin)
Audit log	hr, admin	/api/v1/audit-logs/

7. Before you start

Tools

- Python 3.12, Node.js 20 or newer, Git, and Docker Desktop (only to run Postgres).
- An editor and the DeepSeek harness opened on the project folder.
- An empty Git repo named hrms.

Set up the repo

1. Create the repo with a docs folder.
2. Save this Handbook's sections 1 to 6 as docs/HANDBOOK.md so the agent can read them.
3. Add the AGENTS.md below at the repo root (rename it if your harness uses another context file name).
4. Commit. Then start Phase 0.

AGENTS.md (short version)

```
# AGENTS.md
Project: HRMS (Headcount-style HR system). Read docs/HANDBOOK.md first.
Stack: Django 5 + DRF in backend/, React 18 + Vite + TypeScript in frontend/,
PostgreSQL 16. UI is Tailwind + shadcn/ui only. No FastAPI, no Next.js, no other UI kit.

Rules:
1. Do only the phase named in the prompt. No features from later phases.
2. Every API view has an explicit permission class. Scope get_queryset by role
   (employee = own, manager = own team, hr/admin = all). Never rely on hidden buttons.
3. Business logic goes in services.py per app, not in views or serializers.
4. Balance and request-status changes run in transaction.atomic() with select_for_update().
5. Deactivate employees, never delete. Store leave dates as dates.
6. Secrets in .env, never committed. Keep .env.example current.
7. Write tests for permissions and leave logic. Never edit a test just to make it pass.
8. Every page needs loading, empty and error states. Use shadcn/ui components only.
9. Small typed files, no dead code, no new dependency without saying why.
10. When finished, run the tests and the app, then report: files changed, commands run,
    results, and anything not done. Do not claim something works unless you ran it.
```

How to run each phase

1. Open a fresh agent session. Paste the phase prompt.
2. Read the changes yourself, especially permissions, migrations and anything touching leave balances.
3. Run the app and click through the Done when list. Do not trust the agent's summary.
4. If something breaks, give the agent the exact error and fix one problem at a time.
5. Commit and tag the phase (phase-0, phase-1, ...) so you can roll back.

PHASE 0

Setup

Rough time: half a day

Goal

Django, React and Postgres running together, with the React page talking to the API.

What to build

- backend/ with Django, DRF, psycopg and django-environ; settings read from .env.
- frontend/ with Vite, React, TypeScript, React Router, TanStack Query, Tailwind and shadcn/ui.
- docker-compose.yml that runs only Postgres.
- A health endpoint and a React page that shows its result.

What to do

1. Create the repo, add docs/HANDBOOK.md and AGENTS.md, commit.
2. Open an agent session and paste the prompt.
3. Start Postgres with docker compose up -d, then run the backend and the frontend as the README says.
4. Open the React page and check it shows 'ok' from the API. Tag phase-0.

Prompt for DeepSeek

```
Read AGENTS.md and docs/HANDBOOK.md. Do Phase 0 only.

- backend/: Django 5 project named config, DRF, psycopg, django-environ. Settings read from
  .env (add .env.example). Database is PostgreSQL.
- frontend/: React 18 + Vite + TypeScript, React Router, TanStack Query, Tailwind, and
  shadcn/ui initialised. Vite dev server proxies /api to the Django server.
- docker-compose.yml with one service: postgres:16 (volume for data).
- GET /api/v1/health/ returns {"status": "ok"}, with a pytest-django test.
- React home page calls it with TanStack Query and shows the status in a shadcn Card.
- README with the exact commands to start Postgres, run migrations, start backend and
frontend.

Build nothing else. Run the test and start both servers to check. Report as AGENTS.md rule
10.
```

Done when

- The React page shows 'ok' coming from Django, with Postgres running in Docker.
- The health test passes.

Watch out for

- Secrets hard-coded in settings: they must come from .env.
- CORS config added by the agent: not needed because Vite proxies /api.

PHASE 1

Auth and roles

Rough time: 1-2 days

Goal

People can log in and the app knows their role.

What to build

- Custom User model with email login and a role field: admin, hr, manager, employee.
- Login, logout, me and password-reset endpoints using sessions and CSRF.
- Role permission classes in core/permissions.py.
- React login page, auth context, protected routes, and an app shell whose sidebar changes by role.
- A seed command that creates one admin, one hr, one manager and two employees.

What to do

1. Paste the prompt in a fresh session.
2. Read accounts/models.py and core/permissions.py yourself; everything else depends on them.
3. Log in as each demo user and check the sidebar differs by role.
4. Open a page while logged out and confirm you are sent to login. Tag phase-1.

Prompt for DeepSeek

```
Read AGENTS.md and docs/HANDBOOK.md. Do Phase 1 only (auth and roles).

Backend:
- accounts app with a custom User: email as USERNAME_FIELD, role choices
admin/hr/manager/employee.
  Set AUTH_USER_MODEL before the first migration.
- Session auth with CSRF. Endpoints under /api/v1/auth/: login, logout, me, password-reset
  (console email backend). DRF default permission is IsAuthenticated.
- core/permissions.py with IsAdmin, IsHR, IsManager, IsHROrAdmin and a helper that scopes
querysets by role.
- Management command seed_users: one admin, one hr, one manager, two employees.

Frontend (shadcn/ui only):
- Login page, auth context that calls /auth/me/, ProtectedRoute and RoleRoute components.
- App shell with sidebar and top bar; menu items depend on role; user menu with logout.
- Clean modern look: consistent spacing, light theme, sidebar collapses on mobile.

Tests: login success and failure, me endpoint, each role against a protected test endpoint,
logged-out access is rejected.

Report as AGENTS.md rule 10.
```

Done when

- Four roles log in and see different menus.
- Logged-out users can reach only the login page.
- The permission tests pass.

Watch out for

- Role checks only in React: confirm the API itself returns 403 for the wrong role.
- AUTH_USER_MODEL must be set before the first migration, or fixing it later is painful.

PHASE 2

Teams and employees

Rough time: 2-3 days

Goal

One place for people data: teams, employees, invites and an audit trail.

What to build

- Team and Employee models with CRUD API and role-scoped querysets.
- Creating an employee also creates a login and an invite link to set a password.
- Deactivate instead of delete.
- AuditLog rows written automatically on create, update and delete.
- React pages: employee list (search, filters, pagination), employee form, teams page, accept-invite page.

What to do

1. Paste the prompt in a fresh session.
2. Read each viewset's get_queryset and check it for every role.
3. As HR create an employee, open the invite link from the console, set a password and log in as that employee.
4. As an employee try to open someone else's record; it must fail. Check the audit table. Tag phase-2.

Prompt for DeepSeek

```
Read AGENTS.md and docs/HANDBOOK.md. Do Phase 2 only (teams, employees, audit).

Backend:
- organization.Team (name unique, lead nullable FK to Employee).
- employees.Employee: one-to-one User, first_name, last_name, phone, job_title, team FK,
  manager self-FK, start_date, employment_type, status (active/inactive).
- DRF viewsets at /api/v1/employees/ and /api/v1/teams/. get_queryset scoped by role:
  employee = own record, manager = own team, hr/admin = all. Employees may edit only their
phone;
  hr and admin edit everything. Search and filter by name, team, status. Pagination 20 per
page.
- Creating an employee (in services.py) also creates the User and an invite token (console
email);
  endpoint to accept the invite and set a password.
- Deactivate action, no hard delete. Inactive users cannot log in.
- audit app: AuditLog (actor, action, model, object_id, changes JSON, created_at) written by
  post_save/post_delete signals, with a middleware that stores the current user.
- Extend seed_users to create teams and link employees.

Frontend (shadcn/ui only):
- Employees page: data table with search, team and status filters, pagination, status
badges.
- Employee detail and edit form (react-hook-form + zod). Teams page with create/edit dialog.
- Accept-invite page. Toasts, loading skeletons and empty states.

Tests: permission matrix per role on employees and teams, deactivation blocks login,
invite flow, audit rows created.

Report as AGENTS.md rule 10.
```

Done when

- HR creates an employee who logs in through the invite link.
- Each role sees only what it should, enforced by the API.
- Audit rows exist for the changes.

Watch out for

- An employee editing fields or records they should not: try a PATCH on someone else's record.
- Duplicate audit rows: one action should create exactly one row.

PHASE 3

Time off

Rough time: 3-5 days

Goal

Leave management that replaces spreadsheets: request, approve, balance, calendar.

What to build

- LeaveType, LeaveBalance, LeaveRequest and Holiday models, with a services layer.
- Working-day calculation, balance check, overlap check (in code and in the database).
- Approve, reject and cancel that keep balances correct inside a transaction.
- React pages: my time off, request dialog, approvals queue, team calendar.

What to do

1. Ask the agent to write the tests for timeoff/services.py first and show them to you. Read them.
2. Paste the prompt and let it implement until the tests pass.
3. Test by hand: request, approve as manager, check the balance, cancel, check it comes back.
4. Try to break it: overlapping dates, more days than the balance, end before start, a span over a weekend and a holiday. Tag phase-3.

Prompt for DeepSeek

```
Read AGENTS.md and docs/HANDBOOK.md. Do Phase 3 only (time off). Write tests FIRST for
timeoff/services.py, show them to me, then implement.

Models in the timeoff app:
- LeaveType (name, annual_days, is_paid, requires_approval, carry_over_max)
- LeaveBalance (employee, leave_type, year, allocated, used), unique on employee+type+year
- LeaveRequest (employee, leave_type, start_date, end_date, half_day, days, status
  pending/approved/rejected/cancelled, reviewer, review_comment, created_at)
- Holiday (date, name)
DB constraints: CHECK end_date >= start_date, and a Postgres exclusion constraint
(btree_gist)
that prevents overlapping non-cancelled requests per employee.

services.py (all in transaction.atomic, balance rows locked with select_for_update):
- count_working_days(start, end, half_day): Mon-Fri minus holidays, half day = 0.5
- request_leave, approve_leave, reject_leave, cancel_leave
- Reject balance overruns and overlaps with clear error messages.
- Approve deducts the balance; reject or cancel of an approved request restores it.
- Approver is the employee's manager, or HR if none. Managers act only on their own team.
- Write an audit entry for every state change.

API under /api/v1/: leave-types (hr/admin write), leave-balances, leave-requests with
approve/reject/cancel actions, leave-calendar (approved leave for the user's scope).
Management command allocate_balances for a year. Extend the seed command.

Frontend (shadcn/ui only):
- My time off: balance cards per type and a table of my requests with status badges.
- Request dialog: date range picker, half-day toggle, live day count and balance preview.
- Approvals page for manager and hr with approve/reject and a comment.
- Team calendar: month view showing who is off, colored by leave type.

Report as AGENTS.md rule 10.
```

Done when

- A leave request goes end to end with correct balances.
- Tests cover weekends, holidays, half days, overlaps, overruns and cancel; they pass.
- The calendar shows approved leave.

Watch out for

- Balances drifting: approve then cancel must return the balance to its start value.
- Double approval at the same moment must not deduct twice; check select_for_update is used.

PHASE 4

Polish and demo

Rough time: 2 days

Goal

An app that looks good, has demo data, and is ready to show.

What to build

- Consistent pagination and error messages across the API.
- Loading, empty and error states, toasts and a responsive layout on every page.
- seed_demo command with a believable company.
- Leave types and holidays editable by HR (Django admin is fine).
- docs/PERMISSIONS.md listing every endpoint and the roles allowed.
- README with setup steps. Deployment is optional.

What to do

1. Paste the prompt.
2. Open PERMISSIONS.md and test three risky endpoints by hand with the wrong role.
3. Run seed_demo on a fresh database and walk through the demo script (last page).
4. Optional: deploy the backend and the built frontend, with a managed Postgres, and test again there. Tag phase-4.

Prompt for DeepSeek

```
Read AGENTS.md and docs/HANDBOOK.md. Do Phase 4 only (polish and demo).

1. API: consistent pagination and one error format {"detail": str, "code": str, "fields":
{}}.
2. Frontend: skeletons, empty states, error boundary and toasts on every page; works down to
   375px; consistent spacing and typography with shadcn/ui; accessible form labels.
3. Leave types and holidays manageable by hr/admin (Django admin is acceptable).
4. Management command seed_demo: 3 teams, 20 employees, balances for the current year, and a
   mix of pending, approved and rejected leave requests.
5. docs/PERMISSIONS.md: every endpoint, the roles allowed, how its queryset is scoped.
   Add a test that fails if any view lacks an explicit permission class.
6. README: clear setup, run and seed steps.

Do not add new features. Report as AGENTS.md rule 10.
```

Done when

- Demo data loads in one command and all four roles work.
- PERMISSIONS.md matches the code.
- You can run the full demo script without errors.

Watch out for

- Pages that look fine on desktop and break at phone width.
- A demo that needs fresh data: make sure seed_demo can be run again from scratch.

Later ideas

Only after the MVP is done and working. One feature at a time, each with its own prompt.

- Onboarding and offboarding checklists.
- Reports (headcount by team, leave usage) with CSV export.
- Document uploads on employee profiles.
- Org chart view from the manager links.
- Surveys, public holidays per country, email on leave decisions.

Prompt template for any new feature

```
Read AGENTS.md and docs/HANDBOOK.md. Add ONE feature: <name>.
Phases 0-4 are done. Do not change existing behavior or APIs.
Add models, services.py logic, role-scoped DRF endpoints, React pages (shadcn/ui only),
audit entries and tests. Update docs/PERMISSIONS.md.
Acceptance criteria: <3 to 5 checks you can click through>.
Report as AGENTS.md rule 10.
```

Capstone review: what to prepare

- Demo script (5 minutes): log in as HR, create a team and an employee, accept the invite as that employee, request leave, approve it as the manager, show the balance and calendar, show the audit log.
- Be ready to explain: how role-scoped querysets work, how the leave service avoids double deduction, and why overlap is blocked in Postgres.
- Know what you left out (payroll, recruiting, attendance) and why; it shows scope control.
- Say openly that an AI agent wrote much of the code and that you reviewed permissions, migrations and the leave logic yourself.

When something goes wrong

```
Tests fail: Here is the failing output: <paste>. Find the root cause and fix it without
changing the tests, unless the test itself is wrong. Say which it was.

Permission review: List every endpoint, its permission classes and how get_queryset is
scoped. Flag any endpoint where an employee could read or change another employee's data.

Migration review: Show the SQL for the pending migrations and flag anything destructive.
```